

TUTORIAL SESSIONS



5210COMP:

Software Engineering for Games

Module Leader: Dr Chris Carter

Tutorial 03:

SFML Tutorial #02:

Sprite Game Objects

Important: Finish all tasks in the Tutorial 02 before starting these tasks

Tutorial Overview

This tutorial will help you to develop a GameObject that contains a sprite. In this tutorial you will:

- The **First Task** create a new GameObject that contains a sprite.
- The **Second Task** is to add an instance of the GameObjectWithSprite to the GameState.
- The **Third Task** is to manipulate the GameObjectWithSprite using translation, rotation and scaling functions.
- Additional tasks are for you to work on at home.

Task 1: Look at the Model Solution to Tutorial 02

If you struggled with any of the tasks and syntax from Tutorial 02 – this should indicate that you need to focus on learning some of the core C++ syntax for objects and some further OO programming before you progress.

Step 1: Obtain the Model Solution to Tutorial 02 from Canvas

Download the Model Solution to Tutorial 02 that can be found under the Tutorial 03 Session on Canvas.

Step 2: Analyze and Experiment with the Code base

Have a look at the code for GameStateTest and GameObjectWithText and compare it against your implementation.

Try and modify GameStateTest to see how we can customize the objects.

Try to add multiple instances of the GameObjectWithText to the

Go through this code with your group members and compare each other's implementation.

Today, we will be creating a very similar class, this time to represent Sprites instead of Text objects.

Task 2: Create a GameObject that has a sprite

The Sprite class that I have written in this engine is just a wrapper for the SFML sprite class that you used in Level 4 in Maths and 2D Graphics.

This wrapper class is used to integrate the SFML sprite class into the SEG Engine and provide easy access to the Asset Manager and RenderManager that take care of loading textures and rendering the sprite to the screen.

Step 1: Create a new class that inherits from GameObject

Create a new class that *inherits from GameObject* and call it **GameObjectWithSprite**. Remember to include the `GameObject` class from your **GameObjectWithSprite** class using the concepts from Tutorial 02.

Step 2: Add the required functions

As you did in the previous tutorial when creating the `TextGameObject`, add all the functions required for a class that inherited from `GameObject`. This means looking for the virtual functions in the `GameObject` class.

Use the **override** keyword to ensure you are getting this right!

<https://en.cppreference.com/w/cpp/language/override>

Step 3: Add a Sprite member

Add an instance of the `Sprite` class as private member called `m_Sprite`; you will need to include **Sprite.h** from the Engine (again, not `sf::sprite`!).

Step 4: Initialise m_Sprite

In the `GameObjectWithSprite` `Initialise` function, call `Initialise` on `m_Sprite`. This function takes a number of parameters: filename

- Set the filename parameter to be "sprites/spr_hud_face_raff_0.png".
- Make sure you can locate the sprite in the Image folder in the project so you understand where assets are loaded from.

As with the text class, when you initialise the sprite class, it will automatically be registered with the `RenderManager` and therefore will be rendered to the screen until it is destroyed.

Step 5: Fill in the Get Position functions and the Set Position functions

As our SFML sprite member has a position, we can return this in the two different GetPosition function.

Inside the **GetPosition(float& x, float& y)** function, set the x and y reference parameters' values by calling getPosition().x and getPosition().y on the **m_Sprite** member.

If you are unsure of this, have a look at the implementation of this for the GameObjectWithText in the model solution.

Inside the **sf::vector2f GetPosition()** function, return the m_Sprite member's position using this object's getPosition function.

For the two **SetPosition** functions (one taking two floats, the other taking a 2D vector), use these parameters with m_Sprite's setPosition function, again using function overloading, just like we did with the Text object in Tutorial 02.

Step 6: Consider other useful member functions

So, we have two different GetPosition and Two different SetPosition objects – can you think of any other functions that could be common across all game objects? What about Rotation and Scale? What about Movement (relative changes in position?).

It is common for Base Game Objects in a game engine to have a lot of functions, but bear in mind the more you add, the more you need to override for each object that inherits this interface.

For example, UE4 has a game object called Actor – just have a look at the number of functions it provides!

<https://docs.unrealengine.com/4.27/en-US/API/Runtime/Engine/GameFramework/AActor/>

Discuss and debate what you think should be common to all game objects with your other development group members – you will need to come up with a strategy to handle this for all types of game object – we only have two so far – sprite and text – there are a lot more you might need!

With Position, we have been using the SFML functions of Sprite and Text to help us – what other SFML functions common across these objects could be also make common?

You can find the API for SFML's Text and Sprite objects here:

https://www.sfml-dev.org/documentation/2.5.1/classsf_1_1Text.php

https://www.sfml-dev.org/documentation/2.5.1/classsf_1_1Sprite.php

Task 3: Add an instance of the `GameObjectWithSprite` to the `GameStateTest` `GameState`.

We want to see our new class in action, so we need to create an instance of it inside a game state.

Step 1: Create an instance of the `GameObjectWithSprite` class

Open the `GameStateTest.cpp` file and locate the `Initialise()` function.
Create a variable that is a *pointer to a `GameObjectWithSprite`* (e.g. `GameObjectWithSprite* tsprite`) class and initialise it using the `new` keyword (e.g. `= new GameObjectWithSprite()`).

Step 2: Initialise the `GameObjectWithSprite`

Call the `Initialise` function on the pointer that you created in the previous step (remember, we use `->` to access functions of an object from a pointer).

Step 3: Add the `GameObjectWithSprite` to the `GameObject` list

Call the `push_back()` function on the `m_GameObjects` member variable (as it is not a pointer, we use the `.` syntax to access functions), passing in the pointer to the `GameObjectWithSprite` that you have created.

When a object is added to this gameobject list, it means that its update function will be called once per frame as the game state is updated – have a look through the code of the `GameState` base class to see how this happens.

Task 4: Use translation, rotation, and scale to manipulate the GameObjectWithSprite

As the sprite class is just a wrapper for the SFML sprite class, we have access to all of the SFML sprite functions that you used in the 2D Graphics module from Level 4. Let's use these functions to manipulate the sprite on the screen.

Every GameObject that is registered to a Game state will automatically have the Update() function called every frame until it is destroyed.

Step 1: Move (translate) the sprite

Locate the Update() function of the GameObjectWithSprite class. Experiment with the move() and setPosition() functions on the sprite member object, m_Sprite, to move the sprite around the screen.

It is important that you understand the functions that an SFML sprite gives you, as you will be using these in your Sprite class:

https://www.sfml-dev.org/documentation/2.5.1/classsf_1_1Sprite.php

Step 2: Rotate the sprite

Within the Update() function of the GameObjectWithSprite class. Experiment with the rotate() and setRotation() functions on the sprite member object, m_Sprite, to rotate the sprite as it moves around the screen. What units do we need for rotation – how can we use time to help us?

Step 3: Scale the sprite

Within the Update() function of the GameObjectWithSprite class, experiment with the scale() and setScale() functions on the sprite member object, m_Sprite, to scale the sprite as it moves and rotates around the screen.

Whilst this illustrates the concept, this is not the right place to put this code (Why?). So comment out this code in Update before you proceed.

Step 4: Explore other sprite functions – remember, Initialise is called when we first create the sprite, whilst Update will be called every frame – what functions can we call after we have called Initialise to change the initial state (and therefore properties) of the Sprite?

Come up with a list of functions you might like the Sprite Object to have – and then implement them in the next task!

Additional Task: Make the GameObjectWithSprite generic

You don't want to write a new class every time you want to put a new type sprite on the screen. We should be focusing on creating instances (objects) of the same class, which are different based on the data they are storing. Make the **GameObjectWithSprite** class generic so that you can specify customizable properties on the Sprite.

As with the Text object, you are free to modify the Engine's Sprite class, and in particular, its Initialise function, to make it easier to customize.

Here is an example of how we can then use our Text and Sprite objects in a customizable fashion with our game state – note that when we do not use the Setters, it defaults to a particular set of properties – these should be defined in the constructor.

```
void GameStateTest::Initialise()
{
    //Add a Text Object
    GameObjectWithText* tText = new GameObjectWithText();
    //Add it to the Renderer
    tText->Initialise();

    //Add it to the Game Objects
    this->m_GameObjects.push_back(tText);

    //Add a Text Object
    GameObjectWithText* tText2 = new GameObjectWithText();
    tText2->SetCharSize(psize: 50);
    tText2->SetDisplayText(pstext: "Generic Text");
    tText2->SetColour(pcolour: sf::Color::Blue);
    tText2->SetFontFile(pfont: "fonts/wendy.ttf");
    tText2->SetPosition(ppos: sf::Vector2f(x: 100.f, y: 100.f));

    //Add it to the Renderer
    tText2->Initialise();

    //Add it to the Game Objects
    this->m_GameObjects.push_back(tText2);

    GameObjectWithSprite* tSprite = new GameObjectWithSprite();
    tSprite->SetPosition(ppos: sf::Vector2f(x: 300, y: 300));
    tSprite->Initialise();
    this->m_GameObjects.push_back(tSprite);

    GameObjectWithSprite* tSprite2 = new GameObjectWithSprite();
    tSprite2->SetSpriteFile(psprite: "sprites/spr_pistol_jimmy_0.png");
    tSprite2->SetColour(pcolour: sf::Color::Blue);
    tSprite2->SetScale(pscale: 2.0f);
    tSprite2->Initialise();
    tSprite2->SetPosition(ppos: sf::Vector2f(x: 500, y: 500));
    this->m_GameObjects.push_back(tSprite2);
}
```


This code will produce the following once the APIs for the Sprite and Text objects have been correctly developed.

SEG - Relic Shooter



Additional Task: Make the Game Object that contains both text and a sprite.

Use what you have learned to create a new game object that contains both text and a sprite.

This would be very useful for menu buttons that have text and background images. You could call this class **GameObjectWithSpriteAndText** for example.